A Project Report on

URL SHORTENER APPLICATION

Submitted in partial fulfillment of the requirements for the internship

Domain: Python Programming

Duration: 12-08-2026 to 02-09-2026

Partner: upSkill Campus & UniConverge Technologies Pvt Ltd (UCT)

Submitted To:

Department of Computer Science & Engineering

IILM University

Submitted By:

Prince Maurya

Roll No: 25SCS1003001538

Section: 2CSE7

B.Tech CSE

IILM UNIVERSITY

Greater Noida, Uttar Pradesh, India

2026

1. ABSTRACT

This report documents the design, development, and implementation of a URL Shortener application, completed as part of the industrial internship program with upSkill Campus and UniConverge Technologies (UCT). The project focuses on utilizing Python to solve the real-world problem of managing long, cumbersome URLs by generating unique 6-character short links. The final application integrates an event-driven Graphical User Interface (GUI) using Tkinter and ensures persistent data storage locally via the JSON module, maintaining strict user privacy and avoiding cloud dependencies.

2. INTRODUCTION

2.1 Industry Partner Overview

The internship was conducted in collaboration with UniConverge Technologies Pvt Ltd (UCT), a company operating in the Digital Transformation domain with a focus on Internet of Things (IoT), Cloud Computing, and Machine Learning. The program was facilitated by upSkill Campus to provide practical software development experience.

2.2 Project Objective

The primary objective of this project was to transition from theoretical Python scripting to full-scale software development. This included mastering event-driven programming, file I/O operations, error handling, and delivering a polished desktop application suitable for end-users.

3. SYSTEM ARCHITECTURE & METHODOLOGY

The URL Shortener was developed using a lightweight, offline-first architecture to prioritize speed and privacy.

- **Frontend (GUI):** Built with Python's built-in `tkinter` library. It features input fields for long URLs, output fields for generated short codes, and action buttons for shortening, retrieving, and copying to the system clipboard.
- **Backend Logic:** Utilizes the `string` and `random` libraries to generate a 6-character alphanumeric string. A conditional check loop ensures that no two original URLs receive the same short code.

- **Data Persistence:** Employs the `json` module to serialize the Python dictionary mapping into a local `url_database.json` file. This ensures the data survives application restarts.
- **Validation:** Incorporates robust `try-except` blocks to handle file system access errors, along with string formatting checks to require standard HTTP/HTTPS protocols.

4. RESULTS AND TESTING

The application underwent rigorous end-to-end testing to ensure stability. Validation rules successfully intercepted empty inputs and malformed URLs, triggering user-friendly `messagebox` alerts instead of terminal crashes. Furthermore, clipboard integration functioned seamlessly, allowing users to copy the generated URL directly to the OS clipboard without manual highlighting.

5. CONCLUSION & FUTURE SCOPE

The completion of this internship project significantly enhanced my understanding of the Software Development Life Cycle (SDLC). Building the URL Shortener provided hands-on experience with user interfaces and data storage. Future enhancements for this project could include migrating the local JSON storage to a cloud-based NoSQL database like Firebase, or refactoring the backend into a Flask REST API to deploy the tool as a web application.